Appium interview questions, answered with this framework

Appium + Java + TestNG questions as they come up in SDET and test automation interviews. Every answer has a short explanation, an example, and an **In the framework** pointer to the file in this repository that does it, or the reason it is not used. Links go to github.com/niranjankagri/appium-framework.

Versions: Appium 3.8, UiAutomator2 driver 8.7, Appium Java client 11, Selenium 4.51, TestNG 7.12, Android 15 emulator.

Contents

1. Appium fundamentals (Q1-Q10)
2. Locators (Q11-Q18)
3. Waits and synchronisation (Q19-Q24)
4. Gestures (Q25-Q30)
5. Device and app control (Q31-Q40)
6. Framework design (Q41-Q50)
7. Flaky tests, debugging and reporting (Q51-Q57)
8. Hybrid apps, iOS, CI and devices (Q58-Q64)
9. Scenario questions (Q65-Q70)
10. Quick reference

1. Appium fundamentals

Q1. What is Appium and how does it work?

Appium is an open-source server that automates mobile (and desktop) apps through the **W3C WebDriver protocol**, the same protocol Selenium uses. A test sends HTTP requests (find element, click, ...) through a client library to the Appium server; the server passes them to a **driver** for the platform, which talks to the device's own automation framework. For Android that is UiAutomator2: Appium installs a small server app on the device that runs Google's UiAutomator.

```
Java test — java-client — HTTP/W3C —▶ Appium server —▶ UiAutomator2 driver —▶ device (UiAutomator2 server —▶ app)
```

The app under test is not modified, so the same APK that goes to users is tested.

In the framework: the chain is set up in setup.md and opened in DriverManager.

Q2. What changed in Appium 2 and 3 compared with Appium 1?

- **Drivers and plugins are installed separately** (`appium driver install uiautomator2`); Appium 1 bundled them. The server stays small and drivers update on their own schedule.
- **W3C only**: the old JSON Wire Protocol is gone. Non-standard capabilities need the `appium:` prefix.
- Removed endpoints such as `TouchAction` / `MultiTouchAction`; gestures use **W3C Actions** or `mobile:` commands.
- Appium Inspector and Appium Desktop became separate; the Inspector can run as a server **plugin**.
- Appium 3 continues the cleanup: deprecated endpoints removed, newer Node.js required, plugin and driver APIs modernised.

In the framework: Appium 3.8 with `uiautomator2` installed as a driver and `inspector` as a plugin (setup.md, 1.4); gestures use W3C Actions and `mobile:` commands only.

Q3. UiAutomator2, Espresso, XCUITest: what are they and when do you use which?

Driver	Platform	Works on	Notes
UiAutomator2	Android	any app and the system UI	black box; can leave the app (settings, notifications, permission dialogs)
Espresso	Android	one app, from inside its process	faster and synchronises with the UI thread, but needs the app to be rebuilt with test support and cannot touch other apps
XCUITest	iOS	any app	needs a Mac with Xcode; WebDriverAgent runs on the device

UiAutomator2 is the default for Android because it needs nothing from the app's developers.

In the framework: UiAutomator2 (`UiAutomator2Options` in CapabilitiesFactory). The permission dialog and notification shade tests need it, because both belong to the system, not the app.

Q4. What are desired capabilities? Which ones matter most for Android?

Capabilities are the key-value settings sent when a session starts: which platform, device, app and behaviour. With the Java client, typed **Options** classes replace raw maps.

```
UiAutomator2Options options = new UiAutomator2Options()
        .setDeviceName("Pixel_API_35")
        .setPlatformVersion("15")
        .setApp("/path/app.apk")            // installs when missing or older
        .setAppPackage("com.saucelabs.mydemoapp.android")
        .setAppActivity(".view.activities.SplashActivity")
        .setAppWaitActivity("*")             // the splash screen hands over to MainActivity
        .setAutoGrantPermissions(true)
        .setNoReset(true)
        .setNewCommandTimeout(Duration.ofSeconds(120));
```

`udid` picks one device when several are connected; `newCommandTimeout` ends sessions a crashed test left behind.

In the framework: CapabilitiesFactory, values from config.properties.

Q5. What do `noReset` and `fullReset` do?

Setting	Effect
default (both false)	app data cleared after the session, app stays installed
`noReset=true`	nothing cleared, app not reinstalled: fastest, state carries over
`fullReset=true`	app uninstalled after the session and reinstalled before the next: slowest, cleanest

Many teams combine `noReset=true` with their own reset per test, which gives a clean state without a new session.

In the framework: `noReset=true`, and before each test `terminateApp` + `mobile: clearApp` + `activateApp` (DriverManager.resetApp). Cost: about 2 s instead of 5-10 s for a new session.

Q6. How do you find the appPackage and appActivity of an app?

- `aapt2 dump badging app.apk` prints `package: name=...` and `launchable-activity: name=...`.
- For an installed app in the foreground: `adb shell dumpsys activity activities | grep mResumedActivity`.
- Appium: `driver.getCurrentPackage()` and `driver.currentActivity()`.

In the framework: found with `aapt2 dump badging` on the APK and written to config.properties.

Q7. What is Appium Inspector and how do you use it?

A GUI that starts a session, shows a screenshot of the device next to the element tree (the page source), and lists each element's attributes with suggested locators. Its search tests a locator on the live screen before it goes into code. It replaced the retired Appium Desktop.

In the framework: run as a server plugin, `appium --use-plugins=inspector --allow-cors`, then `http://localhost:4723/inspector`, with the capabilities in setup.md, 3.2. Every id in the screens was checked there or in the page source first.

Q8. What is the difference between an emulator and a simulator? Real device or emulator?

An Android **emulator** emulates the hardware and runs the real Android image. An iOS **simulator** runs iOS frameworks on the Mac's own CPU; it does not emulate the hardware. Emulators are cheap, repeatable and fit CI; real devices are needed for performance, camera, sensors, real networks and manufacturer skins (Samsung, Xiaomi). A common split: most tests on emulators in CI, a smaller set on real or cloud devices.

In the framework: an Android 15 emulator locally and in CI; a real phone works with `-Dudid=<serial>` (setup.md, 2.3).

Q9. What do you need installed to run Appium tests on Windows?

JDK, Node.js, the Android SDK (platform-tools, emulator, a platform and a system image), the `ANDROID_HOME` / `JAVA_HOME` variables, Appium plus the UiAutomator2 driver, and an emulator or phone. Android Studio is optional. `appium driver doctor uiautomator2` checks the setup.

In the framework: the full sequence with a check after each step, and the errors met on the way, is docs/setup.md.

Q10. What is the W3C WebDriver protocol, and why does it matter for Appium?

The standard that defines WebDriver's HTTP API (sessions, elements, actions). Appium follows it, so the Selenium concepts (`findElement` , `click` , waits, Actions) carry over, and the same client libraries work. Things the standard does not cover (install app, set location, clipboard) are exposed as `mobile: <command>` scripts or extension endpoints.

In the framework: standard calls in the screens; `mobile:` commands for the extras, e.g. `mobile: clearApp` , `mobile: swipeGesture` , `mobile: setGeolocation` .

2. Locators

Q11. Which locator strategies does Appium support on Android, and which do you prefer?

`id` (resource-id), `accessibilityId` (content-desc), `androidUIAutomator`, `className`, `xpath`, plus `-android datamatcher` / `viewtag` for Espresso. Preference: **id > accessibility id > UiAutomator > XPath**. Ids are set by developers and fast; accessibility ids also work on iOS and double as an accessibility check; UiAutomator covers text and relationships; XPath walks the whole XML tree and breaks when the layout changes.

In the framework: the order is documented in framework-design.md; `locatorStrategiesFindTheSameElement` in AppiumScenariosTest finds the cart icon all five ways.

Q12. Why is XPath discouraged on mobile?

UiAutomator2 has to serialise the whole screen to XML and search it for every XPath lookup, which is slow on big screens; absolute paths break with any layout change; and XPath on text breaks with translation. It is acceptable as a last resort with a short, attribute-based expression.

In the framework: XPath only appears in the locator-strategies interview test, never in a screen.

Q13. What is `UiSelector` and when do you use `androidUIAutomator`?

A UiAutomator query passed as Java-like text. It can match text, description, class, resource id (with regex), and relationships such as `fromParent` and `childSelector`.

```
AppiumBy.androidUIAutomator("new UiSelector().text(\"Log In\")")
AppiumBy.androidUIAutomator("new UiSelector().text(\"Sauce Labs Backpack\")"
        + ".fromParent(new UiSelector().resourceId(\"...:id/productIV\"))")
```

In the framework: side menu entries have no ids, so SideMenu finds them by text; CatalogScreen.openProduct uses `fromParent` to reach the image next to a product name, because only the image opens the product.

Q14. How do you locate an element that has no id and no text?

In order: an accessibility id; a UiSelector relationship to a neighbour that has one (`fromParent`, `childSelector`); class plus index (`new UiSelector().className("...").instance(2)`); and as a last resort a short XPath. If the element matters to tests, ask developers to add a resource id or content-desc; that also improves accessibility.

In the framework: the product image (shared id `productIV` on every card) is found through its name with `fromParent`.

Q15. What is the accessibility id, and why is it useful on both platforms?

On Android it is `content-desc`, on iOS `accessibilityIdentifier` / `accessibilityLabel`. Developers set it for screen readers, so it is stable, and the same name can work on both platforms, which makes shared page objects possible.

```
tap(AppiumBy.accessibilityId("Ascending order by name"));
```

In the framework: the sort options in CatalogScreen.

Q16. How do you handle an element that is not on screen (in a long list)?

Android only creates views for visible rows (RecyclerView), so the element does not exist until it is scrolled into view. Options: `UiScrollable(...).scrollIntoView(...)` in one UiAutomator call, or a loop of swipes with a check after each.

In the framework: BaseScreen.scrollTo swipes in both directions and stops at the end of the list. UiScrollable is not used because this app's list does not report itself as scrollable (found by trying it).

Q17. How do you check that a locator is right before using it in a test?

Appium Inspector's search (magnifier) on the live screen, or `driver.findElements(locator).size()` in a quick session, or reading `driver.getPageSource()`.

In the framework: each screen was mapped from the page source of the running app before its class was written; the commit history records what was found.

Q18. How do you organise locators?

Inside the screen class that owns them, as `private static final By` constants, with short helper methods (`id("cartBt")` adds the package prefix). Tests never see a locator.

In the framework: every class in screens/.

3. Waits and synchronisation

Q19. Implicit wait or explicit wait?

Explicit. An implicit wait applies to every `findElement`, so "check that something is absent" waits the full timeout, and mixing both gives unpredictable total waits. Explicit waits state what is waited for.

```
new WebDriverWait(driver, Duration.ofSeconds(10))
        .until(ExpectedConditions.elementToBeClickable(locator)).click();
```

In the framework: no implicit wait; BaseScreen waits before every tap, type and read; `isPresent` checks without waiting on purpose.

Q20. What is a FluentWait and when is it better than WebDriverWait?

`WebDriverWait` is a `FluentWait<WebDriver>` with defaults. A `FluentWait` sets the polling interval and ignored exceptions, and can wait on any object, not just the driver.

```
int count = new FluentWait<>(header)
        .withTimeout(Duration.ofSeconds(10))
        .pollingEvery(Duration.ofMillis(200))
        .ignoring(NoSuchElementException.class)
        .until(h -> h.cartCount() == 3 ? h.cartCount() : null);
```

In the framework: `fluentWaitForACustomCondition` in AppiumScenariosTest; custom lambdas in `GeoLocationScreen.showsPosition` and `WebViewScreen.contexts`.

Q21. Why is `Thread.sleep` a bad idea in tests?

A fixed sleep is either too short on a slow device (flaky) or too long on a fast one (slow suite). Waits end as soon as the condition holds.

In the framework: no sleeps in tests or screens; the only pause is a 250 ms poll in `Device.waitForKeyboard`, because Appium has no wait condition for the keyboard.

Q22. How do you know a screen has finished loading?

Wait for one element that only exists when the screen is ready. Put that wait in the screen's constructor, so creating the object is the check.

In the framework: `new CatalogScreen()` waits for the "Products" title, which also covers the splash screen; every screen does the same.

Q23. An action returns before the app is ready. Example?

`activateApp` returns while the splash screen is still starting: `queryAppState` reads `RUNNING_IN_BACKGROUND` for a moment. Wait for the state you need instead of reading it once.

In the framework: `appStateLifecycle` in DeviceApiTest waits for `RUNNING_IN_FOREGROUND` (it failed before that wait was added).

Q24. How do you wait for something that is not an element, like the keyboard?

Poll the API until it returns the expected value, with a timeout.

In the framework: Device.waitForKeyboard polls `isKeyboardShown()`.

4. Gestures

Q25. How do you swipe or scroll in Appium 2/3?

`TouchAction` is gone. Use `mobile:` gesture commands (simple, run on the device) or W3C Actions (any path, any number of fingers).

```
driver.executeScript("mobile: swipeGesture", Map.of(
        "left", 108, "top", 600, "width", 864, "height", 1200,
        "direction", "up", "percent", 0.75, "speed", 1500));
```

In the framework: BaseScreen.swipe.

Q26. What are W3C Actions? Show a tap at coordinates.

A sequence of input-source actions (pointer move, down, pause, up) sent in one request.

```
PointerInput finger = new PointerInput(PointerInput.Kind.TOUCH, "finger");
Sequence tap = new Sequence(finger, 0)
        .addAction(finger.createPointerMove(Duration.ZERO, Origin.viewport(), new Point(x, y)))
        .addAction(finger.createPointerDown(PointerInput.MouseButton.LEFT.asArg()))
        .addAction(finger.createPointerUp(PointerInput.MouseButton.LEFT.asArg()));
driver.perform(List.of(tap));
```

In the framework: `tapAtCoordinates` in AppiumScenariosTest.

Q27. How do you draw or drag along a path?

Press, several timed pointer moves, release. On a canvas there are no child elements to check, so compare element screenshots before and after.

In the framework: DrawingScreen.drawCheckMark and `drawWithAPointerAction`.

Q28. Long press, double tap, pinch?

- Long press: `mobile: longClickGesture` (element or coordinates, duration) or W3C down + pause + up.
- Double tap: `mobile: doubleClickGesture`.
- Pinch/zoom: `mobile: pinchOpenGesture` / `pinchCloseGesture`, or two W3C pointer sequences in one `perform`.

In the framework: not needed by the demo app's screens; the W3C examples above show the building blocks.

Q29. Why might a swipe miss its target?

A fast swipe flings the list, so it travels further than the finger, past the element between two checks. Lists also keep their scroll position when the user returns, so the target can be above, not below.

In the framework: exactly this happened in a full run (`addTwoProducts`); the fix (moderate speed, both directions, end-of-list detection) is in commit "Scroll in both directions" and explained in framework-design.md.

Q30. Why not open a navigation drawer with an edge swipe?

On Android 10+ with gesture navigation, a swipe from the left or right edge is the system **back** gesture, so it leaves the screen (or the app) instead of opening the drawer. Use the menu button, or start the swipe a little inside the edge.

In the framework: found while exploring; recorded in framework-design.md and the device-tests commit.

5. Device and app control

Q31. How do you install, launch, background and close an app?

```
driver.installApp(path);        driver.isAppInstalled(pkg);
driver.activateApp(pkg);        driver.terminateApp(pkg);
driver.runAppInBackground(Duration.ofSeconds(3));
driver.queryAppState(pkg);      // NOT_RUNNING, RUNNING_IN_BACKGROUND, RUNNING_IN_FOREGROUND, ...
```

In the framework: Device (background, restart), `appStateLifecycle` in DeviceApiTest.

Q32. How do you test that the app keeps its state when it goes to the background?

Put it in a known state (product open, item in cart), `runAppInBackground`, then check the same screen and data are there.

In the framework: `stateSurvivesBackground` in MobileTest.

Q33. How do you test screen rotation?

`driver.rotate(ScreenOrientation.LANDSCAPE)` and `driver.getOrientation()`. If the activity is locked to portrait in the manifest, UiAutomator2 refuses ("Is it locked programmatically?"), which is itself the expected behaviour to test. Always rotate back afterwards.

In the framework: `screenStaysPortrait` in MobileTest, with an `@AfterMethod` that restores portrait.

Q34. How do you handle the on-screen keyboard?

`isKeyboardShown()`, `hideKeyboard()`. A keyboard can cover buttons, which then are not found or not tappable. Key events: `driver.pressKey(new KeyEvent(AndroidKey.ENTER))`.

In the framework: `keyboardOpensAndCloses` in MobileTest; `typeWithAndroidKeyEvents` in AppiumScenariosTest.

Q35. How do you handle runtime permission dialogs?

Options: `autoGrantPermissions=true` (grants at install), `mobile: changePermissions` to grant or revoke, or tap the system dialog, which belongs to `com.android.permissioncontroller`. Note: clearing app data also takes the permissions back.

In the framework: PermissionDialog accepts every dialog that appears (the Drawing screen asks twice in a row); `runtimePermissionDialog` revokes location and accepts the dialog.

Q36. How do you mock the GPS location?

`mobile: setGeolocation` (or `driver.setLocation(...)`) with latitude, longitude, altitude. Values come back as floats, so compare with a tolerance.

In the framework: `mockGpsLocation` in DeviceApiTest sets Berlin and reads 52.52 / 13.4049983 on the app's screen.

Q37. How do you work with files and the clipboard on the device?

```
driver.pushFile("/sdcard/Download/x.txt", Base64.getEncoder().encode(bytes));
byte[] back = driver.pullFile("/sdcard/Download/x.txt");
driver.setClipboardText("text");  driver.getClipboardText();
```

In the framework: `pushAndPullAFile` and `clipboardRoundTrip` in DeviceApiTest.

Q38. How do you open the notification shade, and why is it tricky?

`driver.openNotifications()`. It is system UI, outside the app; if a test leaves it open, the next tests cannot see the app. Close it in a `finally` block.

In the framework: `notificationShade` in DeviceApiTest. Its first version had no `finally`; one failure then broke the next two tests.

Q39. How do you read device logs?

`driver.manage().logs().get("logcat")`, or `adb logcat -d -b crash` for crashes. Logcat names the exception and the source line when an app crashes.

In the framework: `readLogcat` in DeviceApiTest; the 7th-product crash was diagnosed with `adb logcat -b crash`.

Q40. How do you get device information during a test?

`mobile: deviceInfo` (model, API level, platform version, screen), `getDeviceTime()`, and the session capabilities (`deviceModel`, `deviceApiLevel`, `deviceScreenSize`).

In the framework: the report header uses the session capabilities (DriverManager.deviceInfo); `deviceInfo` in DeviceApiTest.

6. Framework design

Q41. Describe your mobile framework's architecture.

Tests use screen objects; screens use a base class for waits, taps, typing, scrolling and step logging; a driver manager owns sessions; config comes from a properties file with `-D` overrides; a listener writes the report. Tests read like user stories and hold no locators.

In the framework: framework-design.md.

Q42. Page Object Model for mobile: anything different from web?

Same idea, with mobile specifics: a **component** for the header present on every screen; **device actions** (back, background, rotate, keyboard) in their own class because they are not part of any screen; navigation methods that **return the next screen**; and two methods where the next screen depends on state (guest vs logged-in checkout) instead of guessing.

In the framework: Header, Device, CartScreen (`checkoutAsGuest` / `checkout`).

Q43. One session per test or one session per suite?

One per test is clean but slow (5-10 s each). One per thread with a fast reset between tests gives the same isolation at a fraction of the cost.

In the framework: `ThreadLocal<AndroidDriver>` reused across tests, reset per test, all sessions closed in `@AfterSuite` (DriverManager, BaseTest).

Q44. How would you run mobile tests in parallel?

One device per thread: each thread gets its own session with a different `udid` (and for UiAutomator2 a different `systemPort`), via a `ThreadLocal` driver; TestNG `parallel="classes"` with `thread-count` equal to the number of devices. Or a Selenium Grid / cloud provider that hands out devices.

In the framework: the `ThreadLocal` design supports it; the suite runs on one thread because there is one emulator. To scale: a device list in config mapped to threads, plus `systemPort` per device.

Q45. How do you keep tests independent?

Each test starts from the same state (here: app data cleared, catalog shown), creates the state it needs through the UI, and cleans up anything outside the app (rotation, notification shade, files).

In the framework: `resetApp` before every test; `@AfterMethod backToPortrait`; `finally` blocks in `notificationShade` and `pushAndPullAFile`.

Q46. Where do you keep test data?

In one place, with names that say what the data is for; secrets never in the repository.

In the framework: TestData (the app's own demo users, product names, delivery fee, address and the app's placeholder card), records Address and Card.

Q47. How do you make configuration environment-specific?

A properties file for defaults, overridden by system properties (`-Dkey=value`) from the command line or CI, so the same code runs on a laptop, CI and a device farm.

In the framework: Config.

Q48. How do you compare prices or totals?

Parse to `BigDecimal`, never `double`: 49.99 + 5.99 as doubles is 55.980000000000004.

In the framework: Prices; the checkout test checks order total = cart total + delivery.

Q49. How do you group and select tests?

TestNG groups (`smoke`, `cart`, `mobile`, `interview`, `known-bug`), suite files, and `-Dgroups` / `-Dtest`.

In the framework: groups on every test; testng.xml excludes `known-bug`; known-bugs.xml runs only those.

Q50. Why are the app binaries not in the repository?

Binaries bloat git history and go stale. Pin the version and download it in a script (or from an artifact store in a company).

In the framework: scripts/get-app.sh / `get-app.ps1` download release 2.3.0; `apps/` is in `.gitignore`.

7. Flaky tests, debugging and reporting

Q51. What makes mobile tests flaky, and how do you reduce it?

Timing (animations, splash screens, network), state left by earlier tests, lists that scroll differently, system dialogs, and overloaded devices. Remedies: explicit waits built into screens, a reset per test, cleanup in `finally` / `@AfterMethod`, animations off in CI, and evidence for every failure so the cause is found instead of guessed.

In the framework: each of these met in practice and fixed in its own commit (scroll fling, app state timing, notification shade, ANR); see framework-design.md.

Q52. Should failed tests be retried?

Not blindly: a blanket retry hides real bugs and real flakiness. Retry only for causes outside the app and the test, and make every retry visible.

In the framework: RetryOnDeviceProblem retries once only when Android shows "app isn't responding" (ANR); assertion failures are never retried.

Q53. What is an ANR?

"Application Not Responding": Android's dialog when the app's main thread does not handle input within about 5 s. On a loaded emulator it can be environmental; on a real device it is a performance bug to report. logcat shows `am_anr` with the reason.

In the framework: seen in a full run (load average about 10); logcat said "Input dispatching timed out".

Q54. What evidence do you capture when a test fails?

A screenshot at the moment of failure, a screen recording, the steps, the error with a trimmed stack trace, device logs, and the Appium server log.

In the framework: BaseTest.keepEvidence (screenshot + `startRecordingScreen` / `stopRecordingScreen`), HtmlReportListener; CI also keeps `appium.log`.

Q55. How do you record the screen during a test?

```
driver.startRecordingScreen(new AndroidStartScreenRecordingOptions()
        .withVideoSize("540x1200").withBitRate(1_000_000).withTimeLimit(Duration.ofMinutes(3)));
String base64Mp4 = driver.stopRecordingScreen();
```

Android uses `adb screenrecord` (3-minute limit per file); no ffmpeg needed.

In the framework: recording for every test, kept only for failures, switchable with `recordVideo`.

Q56. How do you build a report without a third-party tool?

A TestNG `IReporter` that writes HTML from the results after the run; steps from `Reporter.log`, evidence from result attributes.

In the framework: HtmlReportListener registered through `META-INF/services`, Steps.

Q57. You find a bug in the app. What do you do with the test?

Report the bug with evidence. Keep a test that asserts the **correct** behaviour, so it fails until the fix and then passes on its own; keep it out of the main run (a group or suite) so the main signal stays clean. Do not change the assertion to match the bug.

In the framework: KnownBugTest: the 7th-product crash, quantity 0, and the cut-off country message.

8. Hybrid apps, iOS, CI and devices

Q58. What is a hybrid app and how do you test its web part?

A native app that shows web content in a WebView. Appium lists contexts (`NATIVE_APP`, `WEBVIEW_<package>`); `driver.context("WEBVIEW_...")` switches to web automation through a Chromedriver matching the device's WebView version (Appium can download it with `--allow-insecure chromedriver_autodownload`). Switch back to `NATIVE_APP` for native elements.

In the framework: `hybridScreenOffersAWebViewContext` in AppiumScenariosTest checks the contexts; switching is left out because it needs a matching Chromedriver on every machine.

Q59. What changes for iOS?

XCUITest driver, a Mac with Xcode, WebDriverAgent signed for real devices, `bundleId` instead of package/activity, locators by accessibility id, `-ios predicate string` or `-ios class chain` (no UiAutomator). Screens can be shared when both apps use the same accessibility ids.

In the framework: Android only (no Mac); the screen layer is where iOS locators would go.

Q60. How do you run Appium tests in CI?

On Linux runners: KVM for a fast x86_64 emulator, an emulator runner action, Appium installed with npm, animations off, and the report kept as an artifact even when tests fail.

In the framework: .github/workflows/tests.yml with `reactivecircus/android-emulator-runner`.

Q61. Emulator in CI or a device cloud?

Emulators: free, repeatable, but one Android flavour and slower. Clouds (BrowserStack, Sauce Labs, LambdaTest, AWS Device Farm): real devices and many models, at a cost. Same tests: only the server URL and capabilities change.

In the framework: `appiumUrl` and the device settings are config keys, so pointing at a cloud is a configuration change plus provider capabilities.

Q62. How do you speed up a mobile suite?

Reuse sessions; reset data instead of reinstalling; skip animations; parallel devices; short smoke group on every push and the full suite less often; record video only when useful.

In the framework: session reuse + `clearApp` reset; `smoke` group (`mvn test -Dgroups=smoke`); `recordVideo=false` option.

Q63. What is `systemPort` and when do you need it?

UiAutomator2 forwards a local port to the server on each device. Two parallel sessions on one machine need different `systemPort` values, or they collide.

In the framework: one device, so the default is used (see Q44 for parallel).

Q64. How do you test on different Android versions?

A matrix: emulators with several API levels in CI (one job each), plus `platformVersion` per run. Watch for version differences such as permission dialogs (Android 13+ media permissions) and gesture navigation.

In the framework: Android 15; `-DplatformVersion` and the CI `api-level` are the knobs.

9. Scenario questions

Q65. A test passes locally but fails in CI. How do you investigate?

Look at the evidence from the CI run (screenshot, video, Appium log) before changing code; compare emulator image, speed and animations; check for timing assumptions (sleeps, missing waits); rerun once to see whether it is consistent.

In the framework: the CI artifact **appium-report** contains the report, failure videos and `appium.log`.

Q66. An element is found but the tap does nothing. Why?

Common causes: the element is not the one that handles the tap (a label next to the tappable image), it is covered (keyboard, dialog), it is still moving (list gliding), or the app is frozen. Check the screenshot, the element's `clickable` attribute, and its parent.

In the framework: product names are not tappable; only the image is (CatalogScreen).

Q67. The app crashes during a test. What do you do?

Read the crash from logcat (`adb logcat -d -b crash`), note the exception, the line and the steps; check whether it reproduces by hand; report it with the video; keep a known-bug test.

In the framework: the 7th-product crash: `ArrayIndexOutOfBoundsException: length=6; index=6` at `ProductCatalogFragment.java:156`, reproduced by hand, documented in KnownBugTest.

Q68. What would you test on a mobile shopping app beyond the happy path?

Validation messages, locked accounts, quantity limits, totals with delivery, background/resume, cold start, rotation, keyboard covering buttons, permissions, offline behaviour, accessibility labels, and very long lists.

In the framework: each of these except offline has a test; three defects came out of them.

Q69. How do you decide what to automate on mobile?

Stable, repeated, high-value flows (login, purchase) and checks that are tedious by hand (sorting, totals, many inputs). Keep exploratory testing for new features and visual polish; keep gesture-heavy or hardware-dependent checks few and focused.

In the framework: functional flows automated; one test per device behaviour.

Q70. How did you build this framework?

In small steps, each a commit: project and config, session handling, setup documentation, screens checked against the live app, tests, fixes for what the tests revealed, known bugs, device tests, the report, flakiness fixes found by full runs, interview scenarios, CI. The commit history shows the order and the reasons.

In the framework: `git log` of the repository.

10. Quick reference

Task	Java client (Appium 3)
Start a session	`new AndroidDriver(url, new UiAutomator2Options()...)`
Find by id / accessibility id	`AppiumBy.id("pkg:id/x")` / `AppiumBy.accessibilityId("label")`
UiAutomator	`AppiumBy.androidUIAutomator("new UiSelector().text(\"X\")")`
Wait	`new WebDriverWait(driver, Duration.ofSeconds(10)).until(...)`
Swipe	`driver.executeScript("mobile: swipeGesture", Map.of(...))`
Tap at a point	W3C `PointerInput` + `Sequence`, `driver.perform(...)`
Back / keyboard	`driver.navigate().back()` / `hideKeyboard()`, `isKeyboardShown()`
Key event	`driver.pressKey(new KeyEvent(AndroidKey.ENTER))`
App state	`activateApp`, `terminateApp`, `runAppInBackground`, `queryAppState`
Clear app data	`driver.executeScript("mobile: clearApp", Map.of("appId", pkg))`
Permissions	`mobile: changePermissions`, `mobile: getPermissions`
Location	`mobile: setGeolocation`
Files / clipboard	`pushFile`, `pullFile`, `setClipboardText`, `getClipboardText`
Screenshot / video	`getScreenshotAs`, `startRecordingScreen`, `stopRecordingScreen`
Contexts	`getContextHandles()`, `context("WEBVIEW_...")`
Logs	`driver.manage().logs().get("logcat")`

Official documentation: appium.io/docs · UiAutomator2 driver · Appium Java client · Appium Inspector · TestNG.